CTPIP

Pipeline Explanation

Document control	
Document ID	CTPIP-PE-001
Project	Clinical Trial Protocol Intelligence Platform (CTPIP)
Version	1.1
Date	21 October 2026
Status	Released

Version	Date	Description of change
1.1	21 October 2026	Initial formal release.

Summary

- Ingestion pipeline: PDF to knowledge graph (7 stages).
- Query pipeline: question to verified answer (10 steps).
- A worked example using real output of the system.
- Local AI pipeline: retrieval (RAG), research agent and explainability check.
- Data stores, job control and failure handling.

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Concept in simple terms 4

1.3 Design guarantees 4

2. Pipeline Overview 4

3. Ingestion Pipeline 5

3.1 Stage 1 - Register 5

3.2 Stage 2 - Extract 5

3.3 Stage 3 - Structure 6

3.4 Stage 4 - Chunk 6

3.5 Stage 5 - Extract facts 6

3.6 Stage 6 - Store 7

3.7 Stage 7 - Resolve and write the graph 7

3.8 Job control and failure handling 8

4. Worked Example 8

4.1 Section and chunk 8

4.2 Claims created from that passage 9

4.3 One claim in full (shortened) 9

5. Data Stores 10

6. Query Pipeline 10

6.1 Validation and support labels 11

6.2 Worked query 12

6.3 Comparison and human review 12

7. Local AI Pipeline 12

7.1 Retrieval (the R in RAG) 12

7.2 Generation 13

7.3 Research agent 13

7.4 Explainability check 13

7.5 Measured results (this computer: 4 GB GPU) 14

8. Pipeline Summary 14

1. Introduction

1.1 Purpose

- This document explains how CTPIP converts protocol PDFs into a knowledge graph and how it answers questions from that graph.
- It is written for readers who do not know the project.

1.2 Concept in simple terms

- Reads each page and notes the headings.
- Cuts the text into small labelled passages.
- Marks every drug, disease, dose and rule.
- Writes one record per fact: what it says, which page, and the exact quote.
- Links related records into a graph.
- At question time: finds the matching records, checks them against the source and answers using only those records.

1.3 Design guarantees

- No fact without a source: every clinical fact carries document, page and quote.
- No guessing: unclear items are flagged, not invented.
- Restartable: progress is saved after each stage, so a crash never loses finished work.
- Offline: rules and a dictionary are used by default; an AI model is optional and always verified.

2. Pipeline Overview

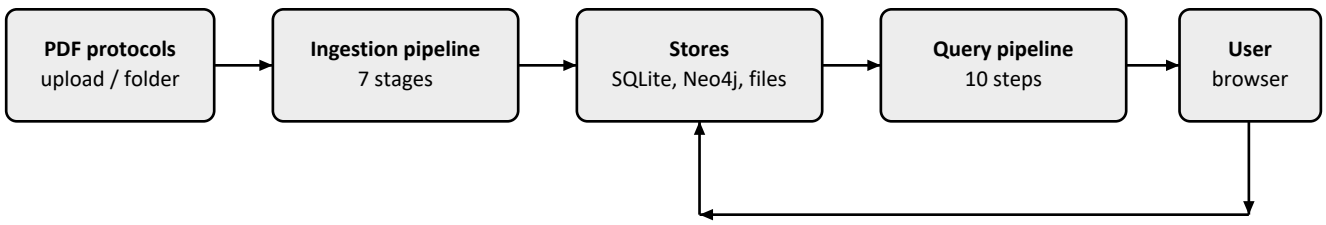


Figure 1: End-to-end view. Uploads flow into the stores; questions travel back through the query pipeline, which reads only from the stores.

Pipeline	Input	Output	Runs in
Ingestion	PDF file	Document record, evidence passages, graph facts	Worker processes and writer thread
Query	Question in plain English	Evidence-grounded, validated answer	API request

3. Ingestion Pipeline

- Seven stages; after each one the progress marker is saved: registered, extracted, analyzed, stored, graphed, done.
- Stages 2 to 5 are CPU-heavy and run in separate worker processes (up to 4 in parallel).
- Stages 6 and 7 write to the databases and run one at a time in a single writer thread to avoid conflicts.

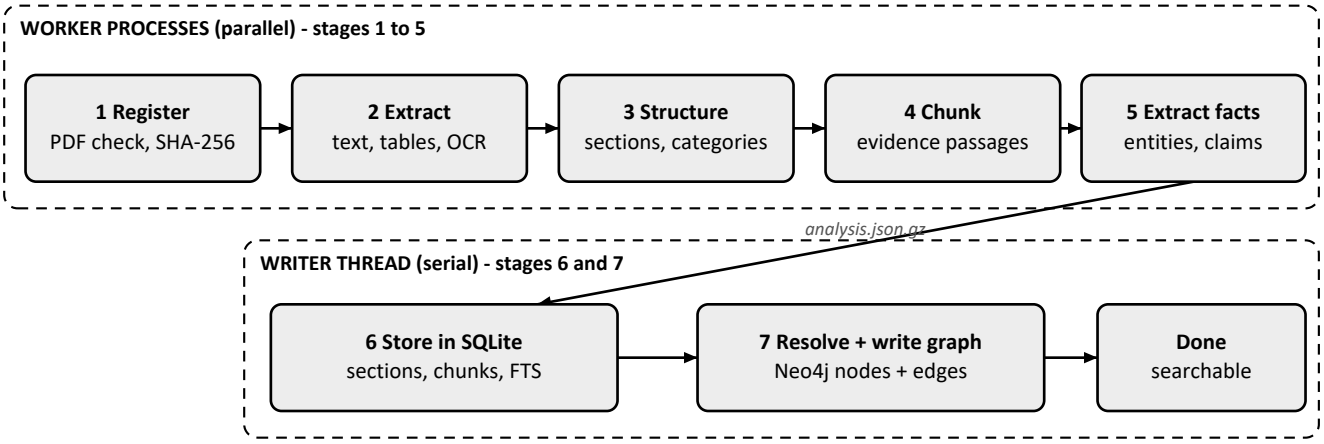


Figure 2: The seven ingestion stages.

3.1 Stage 1 - Register

- Input: a PDF file. Output: a document record and a queued job.
- Checks: starts with `%PDF`, not password-protected, opens correctly, under the size limit (200 MB default).
- Computes a SHA-256 fingerprint; an identical fingerprint is reported as duplicate and processing stops.
- Creates the document id: `d` plus the first 12 characters of the fingerprint.
- Saves a copy in `data/pdfs/` and inserts rows in the `documents` and `jobs` tables (status queued).
- Writes an audit entry.

3.2 Stage 2 - Extract

- Tools: PyMuPDF for text and tables; RapidOCR for scanned pages.

- Output: per page, text lines with font size, bold flag and position, detected tables, method (text or OCR) and confidence.
- Pages with a normal text layer are read directly and keep line coordinates (needed to highlight passages later).
- Pages with fewer than 40 characters are treated as scans and read with OCR (local, offline).
- Tables are detected from ruling lines or a table caption and stored as rows and columns.
- Result saved as checkpoint 1: `extraction.json.gz`; a retry reuses it.

3.3 Stage 3 - Structure

- Output: a tree of sections, each with number, title, level, page range and category.
- First choice: the PDF outline (bookmarks).
- Otherwise: numbered headings, bold or larger short lines, ALL-CAPS lines.
- Printed table-of-contents pages (dot leaders) are ignored as content.
- Categories (for example inclusion_criteria, exclusion_criteria, dosing, treatment_arms, primary_endpoints, statistics) are assigned by ordered keyword rules; the first match wins.
- Sub-sections without a match inherit the parent category (marked inherited); unclear titles stay unknown.

3.4 Stage 4 - Chunk

- Output: chunks, each with page range, bounding boxes, section, type, method and confidence.
- A chunk never crosses a section boundary.
- Eligibility, dosing, arm and endpoint items stay whole; a criterion is never cut and its sub-items stay attached.
- A table becomes one table chunk plus one chunk per row, each row written with its column headings.
- Long paragraphs are split only at sentence ends (about 1,100 characters target, 2,200 maximum); short neighbouring paragraphs are merged.

3.5 Stage 5 - Extract facts

- Title-page data: NCT number, sponsor, phase, version, date, amendment, document type (protocol or SAP).
- Dictionary matching against the curated breast-oncology lexicon: generic names, brands, codes, abbreviations; longest match wins.
- Ambiguous short abbreviations are case-sensitive and need context (for example PR means progesterone receptor only near ER/HER2 wording).
- Document-local abbreviations are learned (from "Trastuzumab deruxtecan (T-DXd; AZD4552)" the document learns that AZD4552 is the same drug).
- Unknown drug-like words become candidates and are not accepted silently.
- Claims are created from section meaning and sentence patterns:

  - a drug in the dosing, arms or synopsis section is INVESTIGATED by the protocol;
  - the same drug in an exclusion criterion becomes a criterion reference, not a treatment;
  - COMBINED_WITH needs a connector such as "+", "plus" or "in combination with";
  - COMPARED_WITH needs "versus" or "compared with";
  - adverse-event statements need reporting language and are never labelled causal;
  - co-occurrence alone never creates a clinical fact.
- Each claim stores up to 4 evidence items: chunk id, pages, verbatim quote and confidence (weighted by section type).
- Optional AI step: proposed claims are kept only if the predicate is allowed, both entities resolve and the quote is found word for word in the chunk.
- Result saved as checkpoint 2: `analysis.json.gz` together with quality warnings.

3.6 Stage 6 - Store

- In one transaction, old rows of the document are deleted and new rows are written to `data/ctpip.sqlite3`.
- Written: sections, chunks (with full-text index), tables, mentions, warnings and document details.
- Replacing instead of appending guarantees no duplicates on re-run.

3.7 Stage 7 - Resolve and write the graph

- Entity resolution, in order of reliability:
  - exact match to the lexicon;
  - abbreviation defined in the same document;
  - stored human decision;
  - unknown drug-like term becomes a provisional entity (only if found in at least 2 passages).
- A provisional term at least 84% similar to a known drug is not merged; it goes to the human review queue.
- Validation: each claim is checked against the schema (allowed relationship, allowed source and target types); invalid claims are rejected and logged as warnings.
- Neo4j write: remove the document's earlier contribution; merge shared canonical entities; create document-owned nodes (Protocol, criteria, arms, doses, Document, sections, chunks, tables, claims); create one relationship per claim.
- Relationship properties: claim_id, source_document_id, source_chunk_id, source pages, supporting_quote, extraction_confidence, validation_status.
- Versions: protocols with the same NCT number are linked to one trial node and ordered by version label or date.

3.8 Job control and failure handling

Situation	Behaviour
Normal run	Job: queued, running, done. Document: queued, processing, processed.
Server stopped mid-way	At next start, running jobs return to the queue and resume after the last completed stage.
Unexpected error	Up to 3 attempts, then status failed with the error text; Retry restarts from the checkpoint.
Bad input (corrupt PDF)	Fails immediately and shows the reason.
Pause	The queue can be paused; running jobs finish.
Delete	Removes graph contribution, search rows, stored PDF and checkpoints; re-orders trial versions.

4. Worked Example

- File: `Sample_Protocol_SAMPLE-PEMBRO-001.pdf` (a made-up 4-page Phase 2 protocol: pembrolizumab plus nab-paclitaxel versus nab-paclitaxel alone).
- All figures below are real output of the system; processing took about 10 seconds.

Stage	Result
1 Register	id d68610bec6b23, 4 pages, status queued.
2 Extract	Text PDF, method pymupdf, confidence 1.0, no OCR, no tables.
3 Structure	21 sections, for example synopsis, primary_endpoints, treatment_arms, inclusion_criteria, exclusion_criteria, statistics.
4 Chunk	30 chunks; each of the 7 inclusion and 7 exclusion criteria is its own chunk.
5 Extract facts	NCT99999999, Phase 2, sponsor Example Oncology Research Institute, version 2.0 (14 March 2026), amendment 1; 79 entities, 110 mentions, 89 claims.
6 Store	21 section rows and 30 chunk rows saved and searchable.
7 Graph	73 entities and 89 relationships written; 0 rejected; 0 warnings.

4.1 Section and chunk

```
section: 4.1  Arm A: Pembrolizumab plus Nab-paclitaxel
category: treatment_arms   detection: heading   page: 2
```

```
chunk id: chk_c02a7c6aca0ab00d   type: paragraph   pages: 2-2
method: pymupdf   confidence: 1.0
text: "Patients in Arm A will receive pembrolizumab 200 mg administered
       intravenously on Day 1 of each 21-day cycle, in combination with
       nab-paclitaxel 100 mg/m2 administered intravenously on Days 1, 8
       and 15 of each 21-day cycle."
bounding box (page 2): x 62.7-532.3, y 626.9-668.7
```

4.2 Claims created from that passage

Claim	From - To	Basis	Confidence
COMBINED_WITH	nab-paclitaxel - pembrolizumab	Connector "in combination with"	0.90
HAS_DOSE_SPECIFICATION	protocol - pembrolizumab 200 mg, Day 1, intravenous	Dose pattern	0.88
HAS_DOSE_SPECIFICATION	protocol - nab-paclitaxel 100 mg/m2, Days 1, 8, 15, intravenous	Dose pattern	0.88
USES_INTERVENTION	Arm A - both drugs	Arm section names the drugs	-
HAS_EXCLUSION_CRITERION (7)	protocol - each exclusion criterion	Item inside exclusion section	0.90
HAS_DESIGN	protocol - open-label, randomized	Title and synopsis wording	0.95

- Claims by type (89 in total): MONITORS_ADVERSE_EVENT 11, ASSOCIATED_WITH_BIOMARKER 9, REQUIRES_ASSESSMENT 9, REFERENCES_CONDITION 8, HAS_INCLUSION_CRITERION 7, HAS_EXCLUSION_CRITERION 7, HAS_SECONDARY_ENDPOINT 5, 12 further types with 33 claims.

4.3 One claim in full (shortened)

```
id: clm_4511b3a59da5b9fb   predicate: COMBINED_WITH
subject: drug:nab-paclitaxel   object: drug:pembrolizumab
method: rule:connector   assertion_type: explicit   confidence: 0.9
evidence: page 2, "...in combination with nab-paclitaxel 100 mg/m2..."
          page 2, "Title: Pembrolizumab plus nab-paclitaxel versus ..."
```

- Resulting graph (default view: 11 nodes, 10 relationships): the Protocol node links to its drugs, cancer subtype, phase, design, setting, primary endpoint and biomarkers.
- Shared nodes such as pembrolizumab are the same node used by other protocols; this enables cross-trial questions.

5. Data Stores

Store	Written in stage	Content
`data/pdfs/`	1	Copy of the original PDF.
`extraction.json.gz`	2	Raw page text, positions, tables, OCR flags (checkpoint 1).
`analysis.json.gz`	3 to 5	Sections, chunks, tables, metadata, entities, claims (checkpoint 2).
SQLite	1, 6, 7	documents, jobs, sections, chunks (+ FTS5), doc_tables, mentions, entity_aliases, warnings, resolution_review, resolution_decisions, audit, queries, eval_runs.
Neo4j	7	Nodes (label KG plus type) and relationships with provenance.

Relationship category	Rule	Examples
clinical	Must carry claim, document, chunk, page and quote.	INVESTIGATES, STUDIES, HAS_PHASE, INCLUDES_ARM, COMBINED_WITH, HAS_DOSE_SPECIFICATION
structural	Document and version organisation.	DESCRIBED_IN, VERSION_OF, AMENDS, PART_OF, SUPPORTED_BY
terminology	Reference knowledge, not protocol evidence.	BELONGS_TO_CLASS, SUBCLASS_OF, CONTAINS_DRUG
mention	Mentioned only; no clinical meaning.	MENTIONS

6. Query Pipeline

- Method: GraphRAG - retrieve real facts first, then summarise only those facts.
- Safety rule: question text never becomes database code; it only fills parameters of pre-written read-only templates.

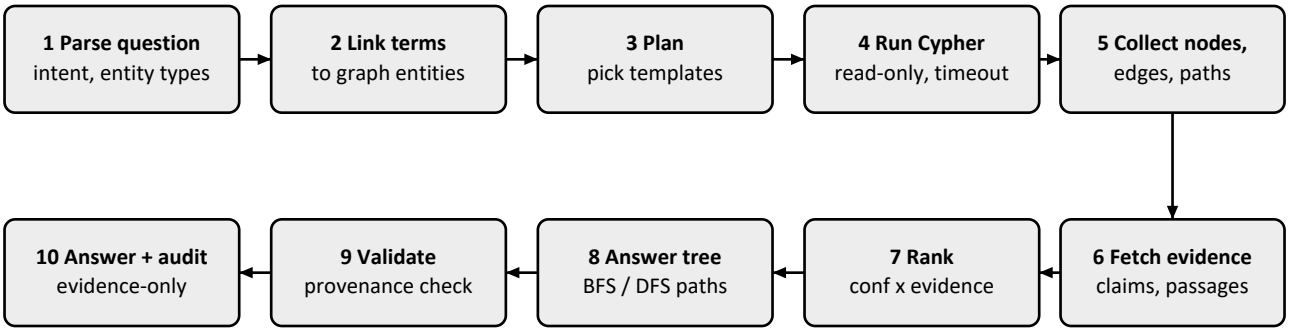


Figure 3: Query pipeline (the code describes 14 finer steps, grouped here into 10).

Step	Name	Description
1	Parse	Regular-expression cues detect the intent (trials for a drug, combination, similar trials, dosing differences, paths).
2	Link terms	NCT ids, then the curated dictionary, then the graph full-text index; each match is verified to exist in the graph.
3	Plan	Intent and entity types select allow-listed templates; an unmappable question returns unsupported with an explanation.
4	Execute	Parameterised Cypher in a read-only transaction; timeout 15 s; limit 400 results.
5	Collect	Chains of real stored nodes and relationships only.
6	Evidence	Claims and supporting passages (page, quote) are fetched.
7	Rank	Confidence x amount of evidence x a penalty that grows with each extra hop.
8	Tree	BFS or DFS tree, visit order with queue or stack state, root-to-answer paths.
9	Validate	Edges exist; clinical edges have claim, document, quote and page; evidence comes from the same document; paths use only returned edges.
10	Answer	Evidence-only text with support labels; plan, evidence and validation are stored in History.

6.1 Validation and support labels

- Validation result: verified, verified with warnings, or no results.
- Support labels: supported by multiple passages, supported by one passage, inferred from a validated path, background literature, reference terminology, mention only, uncertain.
- The answer never adds medical knowledge, never turns a mention into a treatment claim and never implies causality.

6.2 Worked query

- Question: "Find protocols that combine pembrolizumab with chemotherapy".
- Intent: combination.
- Linked entities: pembrolizumab (drug) and chemotherapy (intervention).
- Retrieved: 13 nodes, 22 relationships, 19 reasoning paths.
- Validation: verified.
- Answer: 5 protocols, for example 67 KEYNOTE-355 (NCT02819518), 183 Pembrolizumab + chemotherapy in metastatic TNBC, 142 Pembrolizumab + carboplatin vs carboplatin.
- Time: about 0.5 second when warm; about 10 seconds for the first question after start-up.

6.3 Comparison and human review

- Compare: 2 to 4 protocols are read field by field; each field is same, similar, different, conflicting or unavailable (missing is never treated as exclusion); every value keeps its source passage.
- Review: look-alike drug names (84% or more similar) create review items; a person chooses merge or keep separate; the decision is stored and affected documents are re-graphed from checkpoint 2.

7. Local AI Pipeline

- Purpose: explain protocols and results in plain language and answer free questions, acting as a clinical-trial protocol expert.
- Runs a local model through Ollama (default `qwen2.5:3b`); no data leaves the computer.
- Principle: the model receives numbered sources and may use only those; the answer is then checked against them.

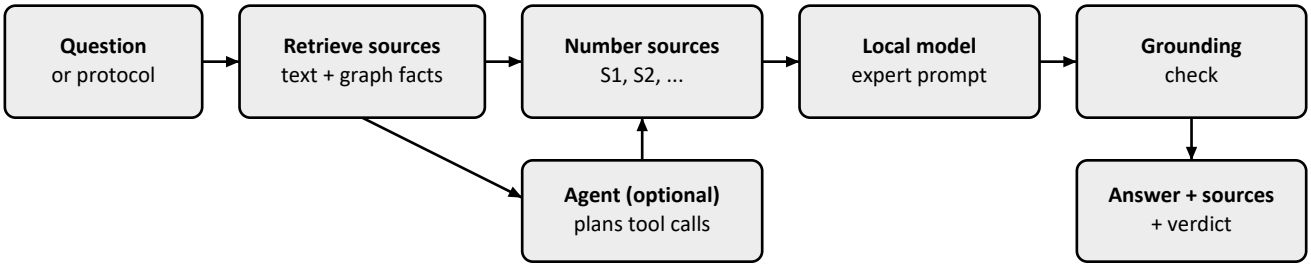


Figure 4: Local AI pipeline.

7.1 Retrieval (the R in RAG)

- Question words are cleaned (question words such as what, how, receive are dropped).
- Drugs and tests named in the question are expanded with their synonyms from the curated dictionary (for example palbociclib: Ibrance, PD-0332991).

- Full-text search (BM25) over all passages; passages in sections that match the topic are ranked higher (eligibility question: eligibility sections; dose question: dosing sections).
- Ranking rewards passages that cover more of the concepts in the question; at most 3 passages per protocol for corpus-wide questions.
- Corpus-wide questions also receive validated graph facts (with quotes) from the query engine.
- Single-protocol questions first receive the protocol's own structured facts for the topic (for example its doses), then passages.
- Each source records why it was chosen (matched words, relevant section, graph fact).

7.2 Generation

- System prompt: senior clinical-trial protocol and clinical-research-management expert for breast cancer, writing for non-medical readers; explain terms; use only the sources; cite [S#] after every fact; never guess doses or numbers; no medical advice.
- Sources are listed one per line ending with their tag, so small models copy the citation format.
- Protocol explanation: built from the protocol brief (structured facts, key section passages, drug-class reference) in ten fixed sections; saved per protocol and model.
- Answers stream to the browser token by token (Server-Sent Events).

7.3 Research agent

- The model plans one step at a time in JSON: find_protocols, protocol_overview, search_text, graph_search or compare (maximum 4 tool calls).
- Each tool result adds numbered sources to a shared pool; repeated steps are blocked; unknown protocol names are reported, not guessed.
- Protocol names are resolved from file names, titles, NCT numbers and sponsor codes, then from the title pages.
- The final answer is written from the pool; every step (reasoning, tool, result) is shown to the user.

7.4 Explainability check

Check	How it works
Cited statements	Citation tags [S#] in each sentence; tags on their own line apply to the paragraph above.
Matched statements	Uncited sentences are compared word by word with every source; the best match above 30% overlap is shown as [~S#].
Invalid citations	Tags that refer to a source that was not given.
Number check	Every dose, percentage or duration in the answer is searched for in the cited and matched sources; trial identifiers are ignored.
Verdict	Well grounded (70% or more supported, no problems), partly grounded (40% or more) or weakly grounded.

7.5 Measured results (this computer: 4 GB GPU)

Task	Model	Time	Result
Explain KEYNOTE-355 (whole protocol)	qwen2.5:3b	about 23 s	38 statements, 53% cited by the model, 82% supported in total, all numbers verified.
Same task	qwen2.5:7b	about 213 s	85% cited by the model; well grounded (slow on a 4 GB card).
Which trials test palbociclib with letrozole and at what dose?	qwen2.5:3b	about 7 s	Two protocols with 125 mg and 100 mg schedules; all 12 numbers verified.
KEYNOTE-355: what treatment, how much, how often?	qwen2.5:3b	about 6 s	Doses of each drug from the protocol's dose facts; 83% supported.
Agent: compare KEYNOTE-355 and PALOMA-3	qwen2.5:3b	about 13 s	Compare tool used; 80% supported.

NOTE: In one test the agent reported PALOMA-3 as Phase 2 because the stored document metadata says Phase 2 while the quoted title says Phase 3. The AI reproduced the source faithfully; the quote shown next to the fact makes such extraction errors visible.

8. Pipeline Summary

Stage	Main tool	Runs in	Saved as
1 Register	SHA-256, PyMuPDF check	API request	documents and jobs rows
2 Extract	PyMuPDF, RapidOCR	Worker process	extraction.json.gz
3 Structure	Layout and keyword rules	Worker process	analysis.json.gz
4 Chunk	Rule-based splitter	Worker process	analysis.json.gz
5 Extract facts	Lexicon, regex, optional AI	Worker process	analysis.json.gz
6 Store	SQLite WAL and FTS5	Writer thread	SQLite tables
7 Graph	Resolver, schema validator, Neo4j driver	Writer thread	Neo4j graph
Query	Planner, templates, Cypher, validator	API request	queries and audit tables
Local AI	Retrieval, local model (Ollama), agent, grounding check	API request (streamed)	audit log; protocol explanations cached